Introduction to Artificial Intelligence

Mihir Rao

Contents

1 Introduction to AI **2**
- 1.1 Rational decisions . . . 2
- 1.2 Agents and environments . . . 2
- 1.3 Methods and applications . . . 3

2 Uninformed Search **4**
- 2.1 Planning and search problems . . . 4
- 2.2 State graphs and search trees . . . 4
- 2.3 Depth-first and breadth-first search . . . 5
- 2.4 Uniform-cost search . . . 5

3 A* Search and Heuristics **6**
- 3.1 Informed search . . . 6
- 3.2 Constructing heuristics . . . 6
- 3.3 Graph search and consistency . . . 6

4 Constraint Satisfaction Problems I **8**
- 4.1 Representation and backtracking . . . 8
- 4.2 Filtering and arc consistency . . . 8
- 4.3 Ordering . . . 9

5 Constraint Satisfaction Problems II **10**
- 5.1 Consistency and structure . . . 10
- 5.2 Local search . . . 11

Lecture 1: Introduction to AI

1.1 Rational decisions

Artificial intelligence can be approached through four goals: making machines think like people, act like people, think rationally, or act rationally. This course focuses on the last: choosing actions that best achieve specified goals. A decision can be rational even if it does not resemble human thinking.

A rational decision maximizes expected utility, a numerical measure of the desirability of an outcome. If h denotes the information available when choosing an action, O is the set of possible outcomes, and $U(o)$ is the utility of outcome o, then for a finite outcome set the rule is

$$a^* \in \arg\max_a \sum_{o \in O} P(o \mid h, a)\, U(o).$$

For each action, weight each possible outcome's utility by its probability, then choose an action with the largest total. Rationality is relative to the goals and information available at decision time: an unlikely bad outcome can occur even when the chosen action had the highest expected utility. The criterion evaluates the decision, not whether its internal reasoning imitates a person.

1.2 Agents and environments

An agent perceives its environment and acts on it. Sensors deliver percepts, and actuators carry out actions. The environment then changes and supplies new percepts, so decisions form a continuing interaction. The nature of the observations, available actions, and environment determines which techniques are appropriate.

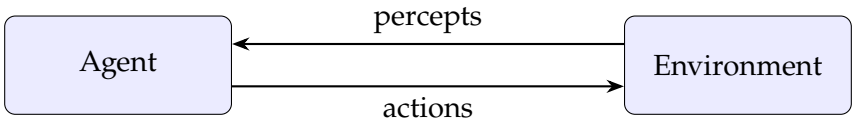


Figure 1: An agent interacts with its environment.

For Pac-Man, the environment is the game, percepts describe what the agent can observe, and actions move Pac-Man through the maze. The agent's objective specifies what counts as a good outcome; seeing the same game does not by itself determine what the agent should prefer.

A REFLEX AGENT chooses from its current percept, possibly using memory, without explicitly evaluating future consequences. A PLANNING AGENT uses a model to consider how candidate actions might change the world and selects a plan in light of its goal. A reflex rule can still be rational when it selects an optimal action for the setting; planning is one way to make decisions, not the definition of rationality.

1.3 Methods and applications

AI systems combine computation and data. Search and planning compare possible actions; constraint satisfaction finds assignments that obey specified rules; adversarial and uncertain settings require reasoning about other agents or uncertain outcomes. Probabilistic models represent uncertainty, decision theory connects uncertainty to utility, and machine learning uses data to improve predictions or decisions.

The same principles appear in different applications:

- **Language:** speech recognition maps audio to text, speech synthesis maps text to audio, and language systems support translation, question answering, and text classification.
- **Vision:** perception turns pixels into information, such as object identities, scene labels, or three-dimensional structure.
- **Robotics:** perception, planning, and control connect decisions to physical actions. The real world can differ from a simulator.
- **Games and logic:** game-playing systems reason over moves; logical systems use deduction, constraint satisfaction, and satisfiability to solve structured problems.

The field has used both knowledge-based and statistical approaches. Recent systems in the 2018 course draw on large datasets, computation, and neural networks; the appropriate representation and objective still depend on the task.

Source. Pieter Abbeel and Dan Klein, Introduction to Artificial Intelligence, Fall 2018, Lecture 1 and Note 1.

Lecture 2: Uninformed Search

2.1 Planning and search problems

A planning agent considers possible consequences before acting. It needs a model of how actions change the world and a goal test. A plan can be computed in advance or recomputed as new observations arrive. Search evaluates possible plans in the model, so its conclusions depend on the model's accuracy.

A search problem specifies a state space, a start state, a goal test, and a successor function. For a state, the successor function gives the available actions, resulting states, and step costs. A solution is a sequence of actions taking the start state to a goal; its cost is the sum of step costs. An optimal solution has minimum cost among all solutions.

A search state is an abstraction of the world: it retains the information needed to determine successors, costs, and goal status. For pathfinding in a maze, a position may suffice. To eat every dot, the state must also record which dots remain. If there are n possible positions and k dots, this representation has at most $n2^k$ states. Adding irrelevant details enlarges the search; omitting a detail that affects future actions or the goal makes the model inadequate.

2.2 State graphs and search trees

In a state-space graph, each state is represented once, and an edge represents an action and its successor. A search tree unfolds the possible plans from the start: its root is the start and each child extends its parent's plan by one action. A tree node represents a path as well as its end state. Thus different nodes can end at the same state, and a cycle in a finite state graph can generate an infinite search tree. Both structures are usually constructed only as needed.

Tree search maintains a fringe of partial plans. It removes a node according to an exploration strategy, checks whether its state is a goal, and otherwise expands it by adding successor plans. The strategy determines which plans are considered first. A goal is returned when selected from the fringe, which is especially important when path costs differ.

We compare strategies by completeness, whether a solution is found when one exists, and optimality, whether the returned solution has least cost. Let b be the maximum branching factor, m the maximum tree depth, and s the depth of the shallowest goal. These tree-search bounds count nodes, including repeated states; assume finite b.

2.3 Depth-first and breadth-first search

Depth-first search (DFS) expands a deepest fringe node first, using a last-in, first-out stack. It can pursue one branch indefinitely, so tree search is not complete for an infinite-depth tree. With finite depth it takes $O(b^m)$ time in the worst case. Its fringe holds at most the unexplored siblings along a root-to-leaf path, requiring $O(bm)$ space. DFS need not find a shallowest or least-cost solution.

Breadth-first search (BFS) expands a shallowest fringe node first, using a first-in, first-out queue. It explores by depth and is complete when branching is finite and a solution exists at finite depth. It finds a shallowest goal, which is optimal when all step costs are the same positive constant. Its time and space are exponential in solution depth; a conservative bound for the usual goal test on removal is $O(b^{s+1})$, because expansions at depth s before the goal can generate depth $s + 1$ nodes.

Iterative deepening runs depth-limited DFS repeatedly with increasing limits. It finds a shallowest goal with the space advantage of DFS. Although it revisits upper levels, for $b > 1$ most nodes lie near the deepest level, so the repeated work changes the asymptotic time by only a constant factor.

2.4 Uniform-cost search

When step costs differ, the fewest actions need not yield the cheapest plan. Uniform-cost search (UCS) uses a priority queue ordered by the cumulative path cost g. It removes a cheapest partial plan, tests for a goal, and expands it if necessary. It explores in increasing cost contours, without using information about the direction of the goal.

Suppose branching is finite, every step costs at least $\epsilon > 0$, and an optimal goal has finite cost C^*. UCS is complete: only finitely many tree nodes can have cost at most C^*, since their depth is at most C^*/ϵ. It is also optimal. If a more expensive goal were removed first, a frontier prefix of an optimal path would have cost at most C^* and would have been selected before that goal. This argument relies on nonnegative step costs and on checking the goal when it is removed, not merely when it is generated. Time and space are exponential in the effective depth C^*/ϵ, with implementation-dependent boundary factors from generating the next level.

Source. Pieter Abbeel and Dan Klein, Introduction to Artificial Intelligence, Fall 2018, Lecture 2.

Lecture 3: A* Search and Heuristics

3.1 Informed search

A heuristic $h(s)$ estimates the remaining cost from state s to a nearest goal. Greedy search expands the node with smallest h, ignoring the cost already paid. It can be fast when the estimate points toward a goal, but can follow a misleading route and need not find a least-cost solution.

A* search instead orders the fringe by $f(n) = g(n) + h(n)$, where $g(n)$ is the cost of the path represented by node n. With $h = 0$ it is uniform-cost search. A goal is tested when removed from the fringe: the first goal generated need not be the cheapest.

A heuristic is admissible if $0 \le h(s) \le h^*(s)$, where h^* is the true least remaining cost. In particular, h is zero at goals. For finite branching and step costs bounded below by a positive constant, A* tree search with an admissible heuristic is complete and optimal. To see optimality, suppose a suboptimal goal of cost $C > C^*$ is on the fringe. Some prefix n of an optimal path is also on the fringe until the optimal goal is removed. Admissibility gives $f(n) \le C^* < C = f(\text{goal})$, so the suboptimal goal cannot be removed first.

3.2 Constructing heuristics

A relaxed problem removes restrictions on actions; its optimal cost is a lower bound on the original cost and therefore an admissible heuristic. In the sliding-tile puzzle, the number of misplaced tiles is a lower bound on moves. The sum of the tiles' Manhattan distances to their goal positions is another: each move changes one tile's distance by at most one. It corresponds to relaxing the restrictions imposed by the other tiles and the blank.

If $h_1(s) \ge h_2(s)$ everywhere, h_1 dominates h_2. A more informative admissible estimate generally avoids expanding nodes with $f > C^*$, though ties and computation cost matter. The maximum of admissible heuristics remains admissible. The exact remaining cost is ideal as a bound, but computing it can be as hard as the original problem.

3.3 Graph search and consistency

Tree search may revisit a state through many paths. Graph search keeps a closed set of expanded states and skips later occurrences. With costs, closing a state reached by a poor path can discard a better path discovered later; admissibility alone does not prevent this in A*.

A heuristic is CONSISTENT if for every edge $s \to s'$ of cost c,

$$h(s) \leq c(s, s') + h(s'), \qquad h(g) = 0 \text{ at goals.}$$

This triangle inequality implies f cannot decrease along a path. It also implies admissibility by summing along a path to a goal. With a consistent heuristic, the first expansion of a state is along a cheapest path to it: a frontier prefix of any cheaper path would have no greater f than that cheaper arrival, and hence would be expanded first. Consequently A* graph search that closes states on expansion is optimal. An alternative for an admissible but inconsistent heuristic is to reopen a state when a cheaper path is found.

Source. Pieter Abbeel and Dan Klein, Introduction to Artificial Intelligence, Fall 2018, Lecture 3.

Lecture 4: Constraint Satisfaction Problems I

4.1 Representation and backtracking

A constraint satisfaction problem (CSP) consists of variables $X_1, \ldots, X_n$, domains D_i of possible values, and constraints on allowed combinations of values. A solution assigns every variable a value and satisfies all constraints. Unlike path planning, the assignment itself is the goal. A constraint may be unary, binary, or involve more variables; it can be stated as a rule or a list of allowed tuples.

For map coloring, variables are regions, domains are colors, and adjacent regions must differ. In a binary CSP the constraint graph has a vertex for each variable and an edge for each pair constrained together. Disconnected components are independent subproblems. With n variables and at most d values each, there can be d^n complete assignments.

Backtracking search performs depth-first search over partial assignments. At each step it picks one unassigned variable and tries its values, rejecting an assignment as soon as it violates a constraint. Assignments commute, so it is unnecessary to branch over all possible orders in which the same variables could be assigned. If no value works, undo the most recent choice and try another. Filtering and ordering can reduce the search without changing its set of solutions.

4.2 Filtering and arc consistency

Forward checking removes from each unassigned neighbor's domain values inconsistent with a new assignment. An empty domain proves failure, but forward checking does not detect every conflict between unassigned variables.

A directed arc $X \to Y$ is arc consistent if every value in D_X has a supporting value in D_Y satisfying the constraint between them. To revise an arc, delete unsupported values from the *tail* domain. To propagate constraints, repeatedly revise arcs; whenever D_X shrinks, recheck arcs from neighbors into X. An empty domain proves that the current branch has no solution. Domains must be restored on backtracking. Arc consistency can be enforced before search and after each assignment.

Arc consistency is not sufficient for global satisfiability. For example, three pairwise unequal variables forming a triangle, each with domain $\{\text{red}, \text{blue}\}$, are arc consistent but cannot all be assigned. Propagation therefore complements rather than replaces search. A basic implementation takes $O(n^2 d^3)$ time in the worst case; more efficient support tracking can improve this bound.

4.3 Ordering

The MINIMUM REMAINING VALUES rule chooses an unassigned variable with the smallest current domain, exposing failure early. The LEAST CONSTRAINING VALUE rule tries the value that eliminates the fewest choices from other variables. The first rule chooses where to branch; the second chooses which branch to try first. Evaluating these choices and propagating constraints has a cost, traded against the search it saves.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 4.

Lecture 5: Constraint Satisfaction Problems II

5.1 Consistency and structure

A CSP is k-consistent if any consistent assignment to $k-1$ variables can be extended to any kth variable. It is strongly k-consistent if it is also j-consistent for every $1 \leq j < k$. Strong n-consistency permits a solution to be built one variable at a time without backtracking, but enforcing higher-order consistency can be expensive.

The constraint graph can make a problem easier. Separate connected components can be solved independently. If n variables split into components of at most c variables, each with domain size at most d, exhaustive search costs roughly $O((n/c)d^c)$ for equal-sized components rather than $O(d^n)$.

A binary CSP whose constraint graph is a tree can be solved in $O(nd^2)$. Choose a root and order parents before children. Process children in reverse order, making each parent-to-child arc consistent by deleting unsupported parent values. Then assign the root and proceed forward, choosing for each child a value compatible with its parent. The backward pass guarantees support for each surviving parent value; once a child has been processed, later revisions do not remove its support. Thus the forward pass needs no backtracking. An empty domain signals failure.

For a nearly tree-structured graph, choose a cutset whose removal leaves a tree or forest. Try each assignment to the cutset, reject inconsistent ones, restrict the remaining domains, and solve the resulting forest. A cutset of size c gives a bound of order $d^c(n-c)d^2$, apart from checking the cutset constraints. Tree decompositions use overlapping groups of variables arranged in a tree, with agreement on shared variables.

5.2 Local search

Local search keeps a single complete assignment and changes it rather than maintaining a fringe of partial assignments. For a CSP, MIN-CONFLICTS selects a variable involved in a violated constraint and assigns it a value that minimizes the number of conflicts. It can be effective on large problems, but can become stuck or cycle and is not generally complete.

More generally, HILL CLIMBING repeatedly moves to a better neighbor and stops when none is better. Local optima and plateaus can prevent it from reaching a global optimum. SIMULATED ANNEALING sometimes accepts a worse move: for objective decrease $\Delta < 0$, a common acceptance probability is $e^{\Delta/T}$. As temperature T falls, downhill moves become less likely. Slow cooling has theoretical convergence guarantees, but escape can still be slow. Genetic algorithms select a population by fitness and create candidates using crossover and mutation. Their usefulness depends on the representation.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 5.